



Project: Tech Stack Advisor — ML-Powered Stack Recommendation App



Lab : Containerize a ML App with Docker and Host it on Hugging Face Spaces



Step 1: Prerequisites

Ensure you have:

-  Python installed
-  UV installed ([Installation](#) | [uv](#))
-  Docker installed (<https://www.docker.com/products/docker-desktop/>)
-  Git installed
-  A Hugging Face account (sign up at <https://huggingface.co/>)
-  Basic familiarity with Python and ML



Tech Stack Advisor – ML App (with Docker & Hugging Face Deployment)

Tech Stack Advisor is a hands-on machine learning project designed to teach you how to build, containerize, and deploy an ML-powered web application using Docker and Hugging Face Spaces.



This project is part of the "**Artificial Intelligence and Machine Learning (AI/ML) with Docker**" course from **School of DevOps**.



What You'll Learn

- Build and train a simple ML model using `scikit-learn`
 - Containerize your app using a Dockerfile
 - Push your Docker image to Docker Hub
 - Deploy the Dockerized app on Hugging Face Spaces (free tier)
-



Project Structure

```
tech-stack-advisor/  
├── app.py           # Gradio web app  
├── train.py         # Script to train and save ML model  
├── requirements.txt # Python dependencies  
├── Dockerfile       # Docker build file (added during the lab)  
├── model.pkl        # Trained ML model (generated after training)  
├── encoders.pkl     # Encoders for categorical inputs (generated after  
training)  
├── LICENSE          # Apache 2.0 license  
└── README.md        # This guide
```



Step 1: Setup and Train Your ML Model

1. Fork the repository: <https://github.com/docker-aiml/tech-stack-advisor>
2. **Clone your own fork of the repository**

```
git clone https://github.com/<your-username>/tech-stack-advisor.git
cd tech-stack-advisor
```

2. Install dependencies

setup virtual environment with uv

```
uv venv --python 3.11
source .venv/bin/activate
```

install dependencies

```
uv pip install -r requirements.txt
```

3. Train the model

```
python train.py
```

This creates:

- `model.pkl`: the trained ML model
- `encoders.pkl`: label encoders for input/output features



Step 2: Run the App Locally (Without Docker)

```
python app.py
```

Visit the app in your browser at:

```
http://localhost:7860
```



Step 3: Test Build the Docker Image Manually

Before you start building an image, make sure that you have trained the model and generated `model.pkl` and `encoders.pkl`

The following process is purely to get an understanding of how to build an image imperatively, using step by step guide, when you are new to building container images. If you are at a pro level, skip this section and proceed directly to using Dockerfile next.

1. Run a dev container with `python:3.11-slim` base image to test build the image with

```
docker run -idt --name dev -p 7860:7860 python:3.11-slim bash
```

2. Copy the source code to the dev container launched above

```
docker cp . dev:/app
```

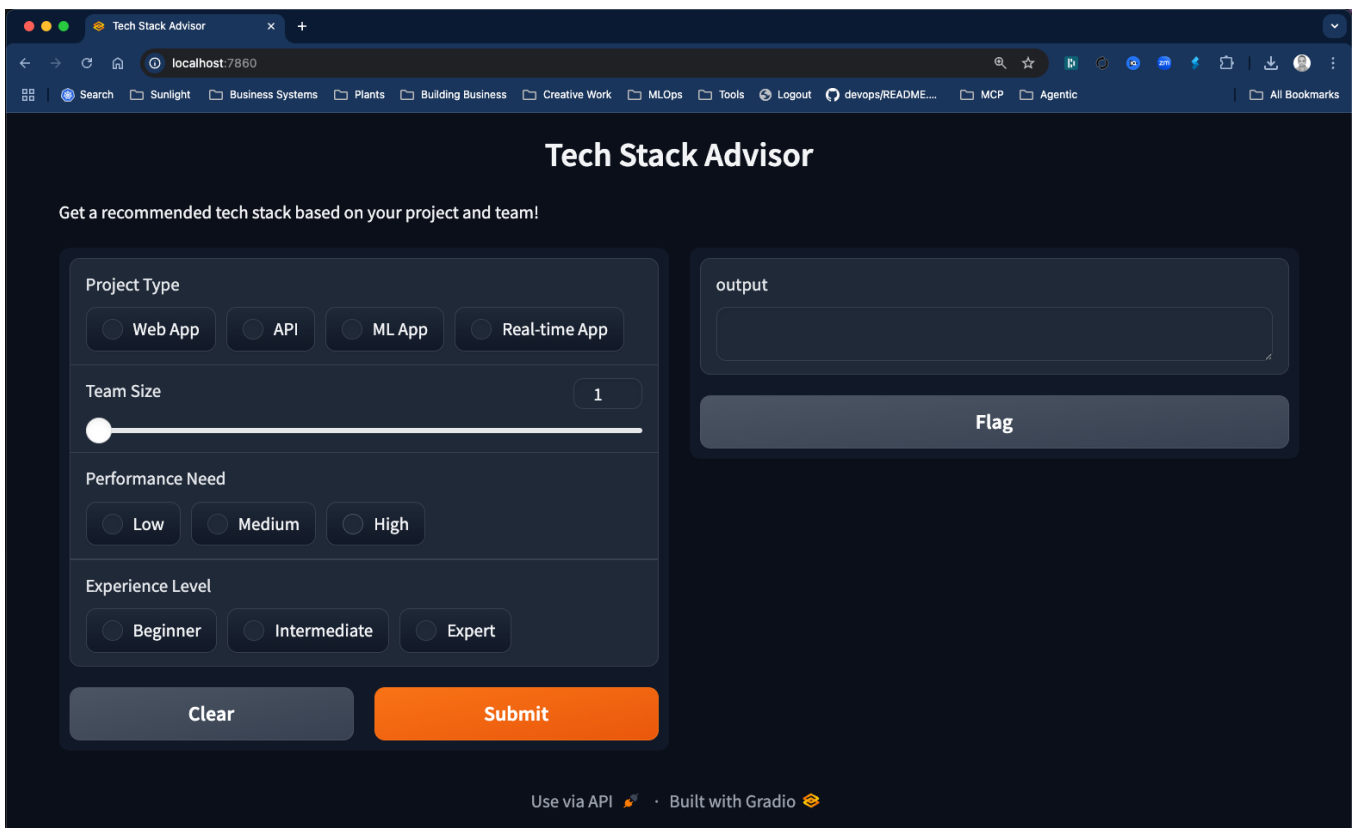
3. Connect to the container and build the app

```
docker exec -it dev bash
cd app
ls
pip install -r requirements.txt
```

4. Test Launch the application

```
python app.py
```

At this time, you should be able to access the app at <http://localhost:7860/> considering you have setup Docker Desktop locally.



5. At this time, you could exit out of the container and take a snapshot of this container to build your first version of container image. This is a imperative approach to build images.

```
# 1. stop the app using "^c" if its still running
# 2. exit out of a container using "exit" or "^d"

# List your containers to find out your dev container which has the app
docker ps

# From your workstation, convert this container into an image as
```

```
docker container commit dev docker.io/xxxxxx/tech-stack-advisor:v1
```

6. You could also publish this image to registry

```
docker image ls
```

```
docker login
```

```
# replace xxxxxx with <your-dockerhub-username>
```

```
docker image push docker.io/xxxxxx/tech-stack-advisor:v1
```

7. Finally delete your dev container created to support this process

```
docker rm -f dev
```

This is a crude approach to building container images, however useful when you are getting started to understand the process, as this is the same process you would convert into code with Dockerfile next.



Step 3: Add Docker Support

Create a file named `Dockerfile` in the root of the project:

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 7860
```

```
CMD ["python", "app.py"]
```

If you observe the Dockerfile, its nothing but the instructions you followed earlier while building the image step by step converted into the code. You could refer to the video lessons from the course to get a better understanding of what is Dockerfile and the instructions present in it.



Step 4: Build a Docker Image with Dockerfile

1. Build the image

```
# replace xxxxxx with <your-dockerhub-username>
docker build -t docker.io/xxxxxx/tech-stack-advisor:v2 .
```

2. Run the container

```
# replace xxxxxx with <your-dockerhub-username>
docker run --name tsa -p 7860:7860 docker.io/xxxxxx/tech-stack-advisor:v2
```

Visit: <http://localhost:7860>

3. Delete the container

```
docker rm -f tsa
```



Step 5: Publish to Docker Hub

1. Compare Docker Images

```
# replace xxxxxx with <your-dockerhub-username>
docker image history docker.io/xxxxxx/tech-stack-advisor:v1
docker image history docker.io/xxxxxx/tech-stack-advisor:v2
```

2. Login to Docker Hub (If not logged in already)

```
docker login
```

2. Tag the image

```
# replace xxxxxx with <your-dockerhub-username>
docker tag docker.io/xxxxxx/tech-stack-advisor:v2 docker.io/xxxxxx/tech-stack-
advisor:latest
```

where, replace xxxxxx with <your-dockerhub-username>

3. Push it

```
# replace xxxxxx with <your-dockerhub-username>
docker push docker.io/xxxxxx/tech-stack-advisor:v2
docker push docker.io/xxxxxx/tech-stack-advisor:latest
```



Step 6: Deploy to Hugging Face Spaces

1. Go to huggingface.co/spaces

2. Click **Create New Space**
 3. Select:
 - **SDK:** Docker
 - **Repository:** Link to your GitHub repo with the Dockerfile
 4. Hugging Face will auto-build and deploy your container.
-

Result

 Your ML app is now hosted at:

```
https://huggingface.co/spaces/<your-username>/tech-stack-advisor
```

Learning Objectives Recap

By the end of this lab, learners will have:

- Trained and saved an ML model using Scikit-learn
 - Built a UI with Gradio
 - Containerized the app with Docker
 - Deployed the container on Hugging Face Spaces using Docker
-